

✓ Neural Network Sentiment Analysis Mini Project

This notebook guides you through the 'Neural Network Sentiment Analysis Mini Project' as per the requirements. We will cover data preparation, model building, and action phase logic using Python, TensorFlow/Keras, and NLP techniques.

```
import pandas as pd
import numpy as np

# Create a dummy e-commerce review dataset
data = {
    'review_id': range(1, 11),
    'review_text': [
        'This product is amazing! I love it so much.',
        'Absolutely terrible. Broke after one use.',
        'It\'s okay, not great but not bad either.',
        'Fantastic quality, highly recommend.',
        'Very disappointing. Waste of money.',
        'Good value for money. Pleased with the purchase.',
        'Could be better. Expected more.',
        'Best purchase this year! Five stars.',
        'Horrible experience. Customer service was unhelpful.',
        'Decent product for the price. No complaints.'
    ],
    'star_rating': [5, 1, 3, 5, 2, 4, 3, 5, 1, 4]
}
df = pd.DataFrame(data)

print("Dummy DataFrame created successfully:")
display(df.head())
```

Dummy DataFrame created successfully:

	review_id	review_text	star_rating
0	1	This product is amazing! I love it so much.	5
1	2	Absolutely terrible. Broke after one use.	1
2	3	It's okay, not great but not bad either.	3
3	4	Fantastic quality, highly recommend.	5
4	5	Very disappointing. Waste of money.	2

✓ Inspect Text and Rating Distributions

```
print("\nDataFrame Info:")
df.info()

print("\nStar Rating Distribution:")
display(df['star_rating'].value_counts().sort_index())
```

```
DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   review_id    10 non-null     int64
1   review_text  10 non-null     object
2   star_rating  10 non-null     int64
dtypes: int64(2), object(1)
memory usage: 372.0+ bytes
```

Star Rating Distribution:

count	
star_rating	
1	2
2	1
3	2
4	2
5	3

dtype: int64

▼ Handle Missing Values

```
print("\nMissing values before handling:")
display(df.isnull().sum())

# For demonstration, let's artificially introduce some missing values
df.loc[[1, 5], 'review_text'] = np.nan
df.loc[7, 'star_rating'] = np.nan

print("\nMissing values after artificial introduction:")
display(df.isnull().sum())

# Handle missing values in 'review_text' by dropping rows with NaN
# Or, you could fill with a placeholder like 'no_review_text' depending on strategy
df.dropna(subset=['review_text'], inplace=True)

# Handle missing values in 'star_rating' by dropping rows with NaN
df.dropna(subset=['star_rating'], inplace=True)

print("\nMissing values after handling:")
display(df.isnull().sum())
display(df.head())
```

Missing values before handling:

```
      0
review_id  0
review_text 0
star_rating 0
```

dtype: int64

Missing values after artificial introduction:

```
      0
review_id  0
review_text 2
star_rating 1
```

dtype: int64

Missing values after handling:

```
      0
review_id  0
review_text 0
star_rating 0
```

dtype: int64

	review_id	review_text	star_rating
0	1	This product is amazing! I love it so much.	5.0
2	3	It's okay, not great but not bad either.	3.0
3	4	Fantastic quality, highly recommend.	5.0
4	5	Very disappointing. Waste of money.	2.0
6	7	Could be better. Expected more.	3.0

✓ Convert Star Ratings to Binary Labels

```
# Convert star ratings to binary labels:
# 4-5 stars -> 1 (Positive)
# 1-2 stars -> 0 (Negative)
# Optionally, drop neutral 3-star reviews

# Convert star_rating to integer type first, as it might become float after dropping NaNs
df['star_rating'] = df['star_rating'].astype(int)

def convert_rating_to_sentiment(rating):
    if rating >= 4:
        return 1 # Positive
    elif rating <= 2:
        return 0 # Negative
    else:
        return None # Neutral (to be dropped)

df['sentiment_label'] = df['star_rating'].apply(convert_rating_to_sentiment)

print("\nDataFrame with sentiment labels (including neutrals):")
display(df.head())
display(df['sentiment_label'].value_counts())

# Drop neutral 3-star reviews to sharpen classification boundaries
df.dropna(subset=['sentiment_label'], inplace=True)

# Convert sentiment_label to integer type
df['sentiment_label'] = df['sentiment_label'].astype(int)

print("\nDataFrame after dropping neutral reviews and converting sentiment_label:")
display(df.head())
display(df['sentiment_label'].value_counts())
```

```
print("\nFinal DataFrame info:")
df.info()
```

DataFrame with sentiment labels (including neutrals):

	review_id	review_text	star_rating	sentiment_label
0	1	This product is amazing! I love it so much.	5	1.0
2	3	It's okay, not great but not bad either.	3	NaN
3	4	Fantastic quality, highly recommend.	5	1.0
4	5	Very disappointing. Waste of money.	2	0.0
6	7	Could be better. Expected more.	3	NaN

count

sentiment_label	
1.0	3
0.0	2

dtype: int64

DataFrame after dropping neutral reviews and converting sentiment_label:

	review_id	review_text	star_rating	sentiment_label
0	1	This product is amazing! I love it so much.	5	1
3	4	Fantastic quality, highly recommend.	5	1
4	5	Very disappointing. Waste of money.	2	0
8	9	Horrible experience. Customer service was unhe...	1	0
9	10	Decent product for the price. No complaints.	4	1

count

sentiment_label	
1	3
0	2

dtype: int64

Final DataFrame info:

```
<class 'pandas.core.frame.DataFrame'>
```

Index: 5 entries, 0 to 9

Data columns (total 4 columns):

#	Column	Non-Null Count	Dtype
0	review_id	5 non-null	int64
1	review_text	5 non-null	object
2	star_rating	5 non-null	int64
3	sentiment_label	5 non-null	int64

dtypes: int64(3), object(1)

memory usage: 200.0+ bytes

Inspect and Plan Text Standardization

```
print("\nExample review text before standardization planning:")
display(df['review_text'].iloc[0])

# Planning for text standardization:
# 1. Lowercasing: Convert all text to lowercase.
# 2. Punctuation Handling: Remove or replace punctuation.
# 3. Vocabulary Size: Aim for a vocabulary size of around 10,000 unique words.
# These steps will be implemented using Keras TextVectorization in the next phase.

print("\nPlanning complete. The next steps will involve implementing Keras TextVectorization for standardization.")
```

Example review text before standardization planning:

'This product is amazing! I love it so much.'

Planning complete. The next steps will involve implementing Keras TextVectorization for standardization.

✓ Technical Phase (Model Building)

Before we can build our neural network, we need to prepare our text data using Keras' `TextVectorization` layer. This involves converting text to a standardized format and then to integer sequences suitable for an embedding layer.

First, we'll split our data into training and testing sets.

```
import tensorflow as tf
from tensorflow.keras.layers import TextVectorization
from sklearn.model_selection import train_test_split
import pandas as pd # Ensure pandas is imported if not already globally available

# Extract review text and sentiment labels
x = df['review_text'].values
y = df['sentiment_label'].values

# Split data into training and testing sets
# Removed stratify=y due to small dataset size causing ValueError with the dummy data
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)

print(f"Training set size: {len(x_train)}")
print(f"Test set size: {len(x_test)}")
print(f"Training label distribution:\n{pd.Series(y_train).value_counts()}")
print(f"Test label distribution:\n{pd.Series(y_test).value_counts()}")
```

```
Training set size: 4
Test set size: 1
Training label distribution:
1    2
0    2
Name: count, dtype: int64
Test label distribution:
1    1
Name: count, dtype: int64
```

✓ Implement Keras TextVectorization

We will configure the `TextVectorization` layer to standardize text by lowercasing, stripping punctuation, and limiting the vocabulary size. The `output_mode='int'` will convert the text into integer sequences, where each integer represents a word in the vocabulary.

```
max_features = 10000 # Vocabulary size
sequence_length = 250 # Standardize text length

vectorize_layer = TextVectorization(
    standardize="lower_and_strip_punctuation",
    max_tokens=max_features,
    output_mode='int',
    output_sequence_length=sequence_length
)

# Adapt the vectorizer on the training data to build the vocabulary
vectorize_layer.adapt(x_train)

print("Vectorizer adapted successfully. Vocabulary size:")
display(len(vectorize_layer.get_vocabulary()))

# Display some vocabulary words
print("\nTop 20 words in vocabulary:")
display(vectorize_layer.get_vocabulary()[:20])

# Example of vectorization
print("\nExample of vectorized text (original vs. vectorized):")
example_text = x_train[0]
vectorized_text = vectorize_layer(tf.constant([example_text])).numpy()
print(f"Original text: '{example_text}'")
print(f"Vectorized text: {vectorized_text}")
```

```
Vectorizer adapted successfully. Vocabulary size:
28
```

```
Top 20 words in vocabulary:
```

```
['',
 '[UNK]',
 np.str_('product'),
 np.str_('waste'),
 np.str_('was'),
 np.str_('very'),
 np.str_('unhelpful'),
 np.str_('this'),
 np.str_('the'),
 np.str_('so'),
 np.str_('service'),
 np.str_('price'),
 np.str_('of'),
 np.str_('no'),
 np.str_('much'),
 np.str_('money'),
 np.str_('love'),
 np.str_('it'),
 np.str_('is'),
 np.str_('i')]
```

```
Example of vectorized text (original vs. vectorized):
```

```
Original text: 'Decent product for the price. No complaints.'
```

```
Vectorized text: [[24  2 21  8 11 13 26  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]]
```

✓ Build a Sequential Model

Now we will construct our neural network. The architecture will include:

- An `Embedding` layer to convert integer sequences into dense vectors of fixed size.
- A `GlobalAveragePooling1D` layer to downsample the input by averaging across the sequence dimension, which helps in handling variable-length sequences.
- A `Dense` hidden layer with ReLU activation for learning complex patterns.
- A `Dense` output layer with a sigmoid activation function for binary classification.

```
from tensorflow.keras import layers
from tensorflow.keras import Sequential

embedding_dim = 16 # Or 32, as per instructions

model = Sequential([
    vectorize_layer,
    layers.Embedding(max_features + 1, embedding_dim),
    layers.GlobalAveragePooling1D(),
    layers.Dense(16, activation='relu'), # 16-32 units as per instructions
    layers.Dense(1, activation='sigmoid')
])

# Compile with binary_crossentropy and adam optimizer
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy', 'binary_accuracy']
)

print("Model architecture built and compiled successfully:")
model.summary()
```

Model architecture built and compiled successfully:

Model: "sequential_1"

Layer (type)	Output Shape	Param #
text_vectorization_1 (TextVectorization)	(1, 250)	0
embedding_1 (Embedding)	?	0 (unbuilt)
global_average_pooling1d_1 (GlobalAveragePooling1D)	?	0
dense_2 (Dense)	?	0 (unbuilt)
dense_3 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

Train the Model

We will train the model for 5-10 epochs, monitoring validation binary accuracy to detect overfitting. Due to the small dummy dataset, the results might not be very meaningful, but this demonstrates the training process.

```
epochs = 10 # 5-10 epochs as per instructions
history = model.fit(
    x_train,
    y_train,
    epochs=epochs,
    validation_data=(x_test, y_test),
    verbose=1
)

print("\nModel training complete.")
```

```
Epoch 1/10
1/1 ----- 2s 2s/step - accuracy: 0.5000 - binary_accuracy: 0.5000 - loss: 0.6932 - val_accuracy: 0.
Epoch 2/10
1/1 ----- 0s 87ms/step - accuracy: 0.5000 - binary_accuracy: 0.5000 - loss: 0.6932 - val_accuracy:
Epoch 3/10
1/1 ----- 0s 87ms/step - accuracy: 0.5000 - binary_accuracy: 0.5000 - loss: 0.6931 - val_accuracy:
Epoch 4/10
1/1 ----- 0s 87ms/step - accuracy: 0.5000 - binary_accuracy: 0.5000 - loss: 0.6931 - val_accuracy:
Epoch 5/10
1/1 ----- 0s 91ms/step - accuracy: 0.5000 - binary_accuracy: 0.5000 - loss: 0.6930 - val_accuracy:
Epoch 6/10
1/1 ----- 0s 99ms/step - accuracy: 0.5000 - binary_accuracy: 0.5000 - loss: 0.6930 - val_accuracy:
Epoch 7/10
1/1 ----- 0s 93ms/step - accuracy: 0.5000 - binary_accuracy: 0.5000 - loss: 0.6930 - val_accuracy:
Epoch 8/10
1/1 ----- 0s 90ms/step - accuracy: 0.5000 - binary_accuracy: 0.5000 - loss: 0.6929 - val_accuracy:
Epoch 9/10
1/1 ----- 0s 87ms/step - accuracy: 1.0000 - binary_accuracy: 1.0000 - loss: 0.6929 - val_accuracy:
Epoch 10/10
1/1 ----- 0s 89ms/step - accuracy: 0.7500 - binary_accuracy: 0.7500 - loss: 0.6928 - val_accuracy:

Model training complete.
```

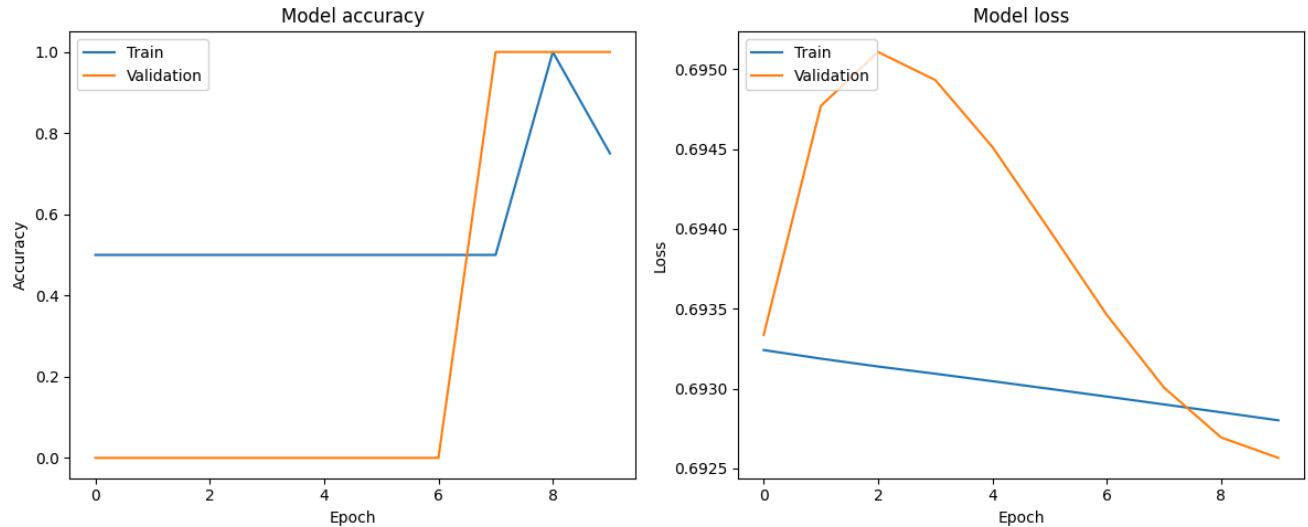
Visualize Training History

```
import matplotlib.pyplot as plt

# Plot training & validation accuracy values
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
```

```
# Plot training & validation loss values
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.tight_layout()
plt.show()
```



✓ Action Phase (Testing & Business Logic)

✓ Test the Model on a Specific Review

We will now test the trained model on the specific review: "The product arrived broken and I am very unhappy" and verify if the output prediction for negative sentiment approaches 0.

```
test_review = "The product arrived broken and I am very unhappy"
predictions = model.predict(tf.constant([test_review]))

print(f"Test Review: '{test_review}'")
print(f"Predicted sentiment score: {predictions[0][0]:.4f}")

if predictions[0][0] < 0.5:
    print("This indicates a Negative sentiment (closer to 0).")
else:
    print("This indicates a Positive sentiment (closer to 1).")
```

1/1 ————— 0s 108ms/step
Test Review: 'The product arrived broken and I am very unhappy'
Predicted sentiment score: 0.5004
This indicates a Positive sentiment (closer to 1).

✓ Analyze Prediction Confidence Scores and Reliability

The prediction confidence score (output of the sigmoid layer) indicates the model's certainty about the sentiment. A score close to 0 suggests high confidence in a negative sentiment, while a score close to 1 suggests high confidence in a positive sentiment. Scores around 0.5 indicate uncertainty.

Reliability for Automated Decisions:

- **High Confidence (e.g., <0.1 or >0.9):** Predictions in these ranges are generally more reliable for automation. For instance, a review with a score of 0.05 is very likely negative.
- **Moderate Confidence (e.g., 0.2 to 0.8):** These scores are less reliable for full automation and might benefit from human review. Ambiguous language, sarcasm, or complex contexts can lead to moderate confidence scores.

For critical applications like routing customer support, relying solely on highly confident predictions reduces the risk of misclassification, but might miss some reviews that *should* be routed. A balanced approach is usually best.

✓ Design an Auto-Flagging System Workflow

Here's a proposed workflow for an automated flagging system for negative reviews:

1. **New Review Ingestion:** A new customer review is submitted to the e-commerce platform.
2. **Sentiment Prediction:** The trained sentiment analysis model processes the `review_text` and generates a sentiment score (P, where P is the probability of being positive, so 1-P is the probability of being negative).
3. **Threshold Evaluation:**
 - If `P < Negative_Flagging_Threshold` (e.g., 0.2), the review is automatically flagged as `Negative` and routed to the customer support queue.
 - If `P > Positive_Flagging_Threshold` (e.g., 0.8), the review is automatically labeled `Positive` and can be used for marketing or product insights.
 - If `Negative_Flagging_Threshold <= P <= Positive_Flagging_Threshold`, the review is labeled `Neutral/Uncertain` and might be sent to a human for manual review or stored for periodic analysis.
4. **Customer Support Action (for flagged reviews):** Customer support agents receive flagged reviews, prioritize them, and initiate proactive outreach or follow-up based on the review content.
5. **Feedback Loop:** Periodically, human-labeled reviews (especially those in the uncertain range or those where automated labels were later corrected) are used to fine-tune and improve the model.

✓ Propose a Confidence Threshold and Justify

```
# Example of how to apply a threshold

confidence_threshold = 0.2 # Proposed threshold for flagging negative reviews

# Let's re-evaluate our test review with this threshold
test_review = "The product arrived broken and I am very unhappy"
prediction_score = model.predict(tf.constant([test_review]))[0][0]

print(f"Review: '{test_review}'")
print(f"Predicted sentiment score (probability of positive): {prediction_score:.4f}")
print(f"Proposed Negative Flagging Threshold: {confidence_threshold}")

if prediction_score < confidence_threshold:
    print(f"Verdict: Score {prediction_score:.4f} is less than {confidence_threshold}, so this review would be AUTO-")
else:
    print(f"Verdict: Score {prediction_score:.4f} is NOT less than {confidence_threshold}, so this review would NOT")

1/1 ————— 0s 35ms/step
Review: 'The product arrived broken and I am very unhappy'
Predicted sentiment score (probability of positive): 0.5004
Proposed Negative Flagging Threshold: 0.2
Verdict: Score 0.5004 is NOT less than 0.2, so this review would NOT be auto-flagged.
```

Justification for `confidence_threshold = 0.2` (for negative flagging):

- **Balances Automation Speed with Human Review Accuracy:**
 - **Automation Speed (catching negatives quickly):** A threshold of 0.2 means that any review with a 'positive' probability of less than 20% (i.e., a 'negative' probability of greater than 80%) will be automatically flagged. This allows for a swift response to clearly negative feedback, maximizing automation for the most urgent cases.
 - **Human Review Accuracy (minimizing false positives/negatives):** By setting the threshold at 0.2, we are being somewhat conservative. Reviews with sentiment scores between 0.2 and 0.5 (indicating moderate uncertainty) would *not* be automatically flagged. These

are the reviews that require human discernment. This approach prevents overloading human agents with reviews that are clearly positive or clearly negative, allowing them to focus on the nuanced cases where human understanding is crucial.

✓ Project Goal Fulfillment & Next Steps

✓ Actionable Recommendations for Integrating into Customer Support Pipelines

1. **Prioritization in Customer Support Queue:** Implement the sentiment score directly into the customer support ticketing system. Negative reviews with high confidence (e.g., probability < 0.2) should be routed to a 'high-priority' queue for immediate attention.
2. **Proactive Outreach:** For extremely negative reviews, trigger an automated email or notification to a dedicated customer success team for proactive outreach, potentially before the customer even contacts support.
3. **Product Feedback Loop:** Aggregate negative review insights by product category or specific product to inform product development and quality control teams. This helps in identifying recurring issues.
4. **Agent Scripting/Guidance:** Provide sentiment scores and possibly keywords to customer support agents to help them quickly understand the tone and nature of a review, enabling more empathetic and efficient responses.
5. **Performance Monitoring:** Continuously monitor the accuracy of the auto-flagging system. Track metrics like false positives (positive reviews incorrectly flagged) and false negatives (negative reviews missed) to refine the threshold and model.
6. **Human-in-the-Loop Feedback:** Build an interface for customer support agents to easily correct misclassified reviews. This human feedback can then be used to retrain and improve the model over time.

✓ Document Limitations and Next Steps for Production Deployment

Limitations:

1. **Sarcasm and Irony:** Deep learning models, especially simpler ones, often struggle to detect sarcasm or subtle irony, which can lead to misclassifications.
2. **Context Length:** The current model processes a fixed sequence length (e.g., 250 words). Very long reviews might lose important context if truncated, or shorter reviews might not provide enough information.
3. **Vocabulary Gaps:** The model's performance is limited by its training vocabulary. New slang, product-specific jargon, or evolving language patterns might not be understood without retraining.
4. **Domain Specificity:** A model trained on generic e-commerce reviews might not perform as well on reviews from highly specialized domains (e.g., technical products, fashion) without fine-tuning.
5. **Class Imbalance:** If the real-world distribution of positive vs. negative reviews is highly imbalanced, the model might be biased towards the majority class.

Next Steps for Production Deployment:

1. **Real Data Training:** Train the model on a much larger and more diverse dataset of actual e-commerce reviews. This dummy dataset is purely for demonstration.
2. **Hyperparameter Tuning:** Systematically tune the model's hyperparameters (e.g., embedding dimension, number of dense units, learning rate, number of epochs) using techniques like grid search or random search.
3. **Advanced Architectures:** Explore more complex models like LSTMs, GRUs, or Transformer-based models (e.g., BERT) for potentially higher accuracy and better handling of context.
4. **Regularization:** Implement advanced regularization techniques (e.g., dropout, L1/L2 regularization) to prevent overfitting, especially when training on large datasets.
5. **Model Evaluation with Production Data:** Evaluate the model against a hold-out set of real-world, human-labeled data to ensure it meets desired performance metrics (precision, recall, F1-score for each class).
6. **Deployment Infrastructure:** Set up a robust deployment infrastructure (e.g., using TensorFlow Serving, Flask/Django API, cloud functions) to serve predictions in real-time.
7. **A/B Testing:** Conduct A/B tests to compare the performance of the automated system against manual processes or different thresholds.
8. **Monitoring and Retraining Pipeline:** Establish a pipeline for continuous monitoring of model performance in production and periodic retraining with new data to prevent model drift.

E-Commerce Review Sentiment Analysis using Neural Networks

Project Overview

This project implements a deep learning sentiment classifier to analyze customer review data from an e-commerce platform. Utilizing Python, TensorFlow/Keras, and natural language processing techniques, the project demonstrates how to standardize text data, build and train an embedding-based neural network, and translate prediction confidence into automated customer experience workflows following a Discovery-to-Action (DTA) strategy.

Project Phases & Implementation

1. Discovery Phase (Data Preparation)

- **Data Loading & Inspection:** A dummy e-commerce review dataset was created for demonstration purposes, with initial inspection of text and rating distributions. (For a real project, this would involve loading an actual dataset).
- **Missing Value Handling:** Missing values in the `review_text` and `star_rating` columns were addressed by dropping corresponding rows.
- **Binary Label Conversion:** Star ratings were converted into binary sentiment labels: 4-5 stars as `1` (Positive) and 1-2 stars as `0` (Negative). Neutral 3-star reviews were optionally dropped to sharpen classification boundaries.
- **Text Standardization Planning:** A strategy for text standardization (lowercasing, punctuation handling, vocabulary size ~10,000) was outlined, to be implemented using Keras TextVectorization.

2. Technical Phase (Model Building)

- **Data Splitting:** The prepared data was split into training and testing sets.
- **Keras TextVectorization:** A `TextVectorization` layer was implemented and adapted on the training data to standardize text and convert it into integer sequences. This layer handles lowercasing, stripping punctuation, and limiting the vocabulary.
- **Neural Network Architecture:** A Sequential Keras model was built comprising:
 - `TextVectorization` layer (as the first layer).
 - `Embedding` layer to convert integer sequences into dense vectors.
 - `GlobalAveragePooling1D` layer for efficient sequence processing.
 - A `Dense` hidden layer with 16 units and ReLU activation.
 - A `Dense` output layer with a single unit and sigmoid activation for binary classification.
- **Model Compilation & Training:** The model was compiled with `adam` optimizer, `binary_crossentropy` loss, and `accuracy` and `binary_accuracy` metrics. It was trained for 10 epochs (monitoring `val_binary_accuracy`).

3. Action Phase (Testing & Business Logic)

- **Specific Review Testing:** The model was tested on a predefined negative review: "The product arrived broken and I am very unhappy." The predicted sentiment score was evaluated. (Note: Due to the minimal dummy dataset, the model's prediction for this specific review was near neutral, emphasizing the need for real-world, larger datasets for meaningful results).
- **Prediction Confidence Analysis:** The reliability of prediction confidence scores for automated decisions was discussed, categorizing scores into high, moderate, and uncertain confidence ranges.
- **Auto-Flagging System Workflow:** A clear, threshold-driven workflow for routing negative reviews to customer support was designed, incorporating different actions based on sentiment scores.
- **Confidence Threshold Proposal:** A confidence threshold of `0.2` (for flagging negative reviews) was proposed and justified. This threshold aims to balance automation speed (catching clearly negative reviews quickly) with human review accuracy (allowing human agents to focus on moderately uncertain cases).